



PDF Download  
3569420.pdf  
22 February 2026  
Total Citations: 47  
Total Downloads:  
2195

Latest updates: <https://dl.acm.org/doi/10.1145/3569420>

RESEARCH-ARTICLE

## Side-channel Analysis of Lattice-based Post-quantum Cryptography: Exploiting Polynomial Multiplication

CATINCA MUJDEI, KU Leuven, Leuven, Vlaams-Brabant, Belgium

LENNERT WOUTERS, KU Leuven, Leuven, Vlaams-Brabant, Belgium

ANGSHUMAN KARMAKAR, KU Leuven, Leuven, Vlaams-Brabant, Belgium

ARTHUR BECKERS, KU Leuven, Leuven, Vlaams-Brabant, Belgium

JOSE MARIA BERMUDO MERA, KU Leuven, Leuven, Vlaams-Brabant, Belgium

INGRID M R VERBAUWHEDE, KU Leuven, Leuven, Vlaams-Brabant, Belgium

Published: 27 March 2024  
Online AM: 04 November 2022  
Accepted: 02 October 2022  
Revised: 07 August 2022  
Received: 01 April 2022

[Citation in BibTeX format](#)

Open Access Support provided by:

KU Leuven

# Side-channel Analysis of Lattice-based Post-quantum Cryptography: Exploiting Polynomial Multiplication

CATINCA MUJDEI, LENNERT WOUTERS, ANGSHUMAN KARMAKAR,  
ARTHUR BECKERS, JOSE MARIA BERMUDO MERA, and  
INGRID VERBAUWHUDE, imec-COSIC KU Leuven, Belgium

Polynomial multiplication algorithms such as Toom-Cook and the Number Theoretic Transform are fundamental building blocks for lattice-based post-quantum cryptography. In this work we present correlation power-analysis-based side-channel analysis methodologies targeting *every* polynomial multiplication strategy for *all* lattice-based post-quantum key encapsulation mechanisms in the final round of the NIST post-quantum standardization procedure. We perform practical experiments on real side-channel measurements, demonstrating that our method allows to extract the secret key from all lattice-based post-quantum key encapsulation mechanisms. Our analysis shows that the used polynomial multiplication strategy can significantly impact the time complexity of the attack.

CCS Concepts: • **Security and privacy** → **Side-channel analysis and countermeasures**; *Public key encryption*;

Additional Key Words and Phrases: Post-quantum cryptography, side-channel analysis, number-theoretic transform, Toom-Cook multiplication

## ACM Reference format:

Catinca Mujdei, Lennert Wouters, Angshuman Karmakar, Arthur Beckers, Jose Maria Bermudo Mera, and Ingrid Verbauwhe. 2024. Side-channel Analysis of Lattice-based Post-quantum Cryptography: Exploiting Polynomial Multiplication. *ACM Trans. Embedd. Comput. Syst.* 23, 2, Article 27 (March 2024), 23 pages. <https://doi.org/10.1145/3569420>

## 1 INTRODUCTION

The advent of quantum computers and their impact on the security of classical **public-key cryptography (PKC)** has increased research interest in **post-quantum cryptography (PQC)**. The majority of the PQC research is being conducted in the context of the PQC standardization procedure led by the **National Institute of Standards and Technology (NIST)** [43]. NIST recently

This work was supported in part by CyberSecurity Research Flanders with reference number VR20192203, the Research Council KU Leuven (C16/15/058), the Horizon 2020 ERC Advanced Grant (101020005 Belfort) and SRC grant 2909.001. This work was supported in part by the Research Council KU Leuven grant C14/18/067 on Cryptanalysis of post-quantum cryptography. Angshuman Karmakar is funded by FWO (Research Foundation - Flanders) as junior post-doctoral fellow (contract number: 203056/1241722N LV).

Authors' address: C. Mujdei, L. Wouters, A. Karmakar, A. Beckers, J. M. Bermudo Mera, and I. Verbauwhe, imec-COSIC KU Leuven, Kasteelpark Arenberg 10 - bus 2452, 3001 Leuven, Belgium; emails: catinca\_mujdei@yahoo.ca, {lennert.wouters, angshuman.karmakar, arthur.beckers, jose.bermudo, ingrid.verbauwhe}@esat.kuleuven.be.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2023 Association for Computing Machinery.

1539-9087/2024/03-ART27 \$15.00

<https://doi.org/10.1145/3569420>

requested more research into the physical security of the PQC implementations as part of the currently ongoing final round of the standardization process [2].

New cryptographic primitives are first assessed cryptanalytically to ensure their theoretical security [3, 4, 20]. A real-world implementation of a secure cryptographic primitive can, however, still be vulnerable to implementation or physical attacks. Side-channel attacks are considered passive physical attacks in which the adversary is able to acquire side-channel information (e.g., power consumption, **electromagnetic (EM)** radiation, execution time, etc.) that is unintentionally produced by the implementation. This side-channel information can be exploited to extract secret information [35].

Lattice-based hard problems have a long history of being used for cryptographic constructions, starting with Ajtai's seminal work on the short integer solution hard problem [1], and later with the introduction of NTRU [25] and the **Learning With Errors (LWE)** hard problem [49]. However, compared to PKC primitives such as RSA [51] and **elliptic-curve cryptography (ECC)** [34, 41], lattice-based cryptography received little attention in the context of physical security.

The current body of work related to side-channel analysis of PQC can be divided into two major classes. The first class of attacks uses side channels to perform **chosen ciphertext attacks (CCAs)**. Multiple works demonstrated that LWE and **Learning With Rounding (LWR)** schemes such as Kyber and Saber are vulnerable to CCA-based side-channel attacks by exploiting leakage caused by the **Fujisaki-Okamoto (FO)** transform [12, 48, 55]. NTRU-based schemes using the FO transform or the Saito-Xagawa-Yamakawa transform have also been shown to be vulnerable to CCA-based side-channel attacks [47]. Most of these attacks use a purposefully crafted ciphertext, such that the decrypted message depends on a single secret coefficient. By varying the ciphertext, the secret coefficients can be retrieved using statistical tests (e.g., the  $t$ -test). On average, a few thousand traces are sufficient to recover the full private key. Vulnerability to CCA side-channel attacks has been demonstrated for all lattice-based schemes. However, the ciphertexts used in most CCA-based attacks will not decrypt to a valid message and will therefore result in decryption failures. Limiting the maximum number of allowed decryption failures can prevent these attacks. For example, a device can impede CCA-based side-channel attacks by deleting all key material after a certain amount of decryption failures have been observed.

The second class of attacks targets the private key directly. This class of attack can use valid ciphertext inputs that do not lead to decryption failures. Primas et al. targeted the **Number Theoretic Transform (NTT)** using profiled single trace attacks [46]. Pessl and Primas later expanded on these attacks in [44]. NTRU, being the oldest variant of the lattice-based finalists, has thus far received the most attention when it comes to physical side-channel attacks. Huang et al. attacked the polynomial multiplication of NTRU Prime using **correlation power analysis (CPA)**, horizontal CPA, template attacks, and **simple power analysis (SPA)** [27]. Earlier works by Lee et al. [37] targeted NTRU using CPA, while Silverman and Whyte [53] performed timing attacks on NTRU-Encrypt. Note that several variants, parameter sets, and implementations exist for NTRU; hence, the listed papers do not all target the most recent variant. In contrast, in this work we focus on the schemes, and their reference implementations, that are currently in the final round of the NIST PQC competition.

Interestingly, the current body of work related to the side-channel security of PQC is focused on CCA attacks and more advanced profiled attacks. In this work we investigate the applicability of classical non-profiled CPA approaches. Specifically, we provide a side-channel analysis of the polynomial multiplication implementations used by *all* lattice-based NIST PQC KEM finalists.

Concretely, the contributions of this article can be summarized as follows:

- We propose a non-profiled side-channel attack methodology targeting all the different polynomial multiplication algorithms used in lattice-based cryptography. We categorically show

how these algorithms can be attacked using CPA. Our techniques are generic for the different multiplication algorithms.

- We demonstrate practical side-channel attacks using real power traces on lattice-based KEMs in the final round of the NIST standardization procedure, i.e., NTRU-KEM [16], Kyber [7], and Saber [10]. For each of these schemes there are multiple implementations relying on different multiplication algorithms. We exhaustively demonstrate our attack on all the reference implementations of these schemes.
- We compare the complexity of attacking the different schemes, multiplication methods, and parameter choices using CPA. We show how the different multiplication strategies can have a significant impact on the data and time complexity of the attack.

## Outline

Section 2 provides background information on the studied lattice-based cryptosystems as well as the different strategies used for polynomial multiplication. In Section 3, we discuss how Toom-Cook-based polynomial multiplication can be attacked using CPA. Similarly, in Section 4, we apply our techniques to the NTT. In Section 5, we state practical results, demonstrating key recovery using real-world side-channel measurements on all lattice-based KEMs in the final round of the NIST standardization procedure. Finally, in Section 6, we summarize and discuss our results, as well as suggest potential countermeasures.

## 2 PRELIMINARIES

This section provides general notation and background information on lattice-based KEMs. We also describe the Number Theoretic Transform and Toom-Cook polynomial multiplication strategies that are used by these lattice-based KEMs.

### 2.1 Notations

Throughout this article, we denote by  $\mathbb{Z}_q$  the ring of integers modulo  $q$  for a positive integer  $q$ . We consider rings of the form  $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ , with  $n$  a power of 2, of which the elements are denoted by lowercase letters, sometimes considering the variable  $x$  implicit; if the variable  $x$  of a polynomial  $a(x) \in R_q$  is specified, then we denote its  $i$ th coefficient by  $a_i$ . Vectors and matrices are written in bold lowercase and bold uppercase, respectively. Given a vector  $\mathbf{v}$ , we write the mean value of its coefficients as  $\bar{\mathbf{v}}$ . Sampling an element  $x$  from a distribution  $\chi$  over some domain  $\mathcal{D}$  is denoted by  $x \leftarrow \chi(\mathcal{D})$ .

### 2.2 Lattice-based Cryptosystems

The lattice-based KEMs in the final round of the NIST PQC competition are based on two constructions: first, Kyber [7] and Saber [21], which are both variants of the **public-key encryption (PKE)** scheme attributed to **Lyubashevsky, Peikert, and Regev (LPR)** [38], and second, NTRU-like schemes [16], which were first proposed by Hoffstein et al. [25]. Specific transforms are applied to these schemes to transform them from chosen-plaintext attack-secure PKEs to CCA-secure KEMs.

We briefly cover these schemes here and refer the reader to the original submission documents (e.g. [7, 10, 15]) for more details.

**2.2.1 LPR Encryption Scheme.** Regev's LWE problem [49, 50] entails solving a system of “noisy” linear equations: given a fixed secret  $\mathbf{s} \in \mathbb{Z}_q^k$ , an error  $e \in \mathbb{Z}_q$ , and a uniformly random element  $\mathbf{a} \in \mathbb{Z}_q^k$ , the goal is to recover  $\mathbf{s}$  from  $(\mathbf{a}, b = \mathbf{a} \cdot \mathbf{s} + e)$ . Secret and error sampling for LWE-based schemes is typically done over a centered binomial distribution  $\beta_\mu$  for a positive even integer  $\mu$  [6]. Samples of  $\beta_\mu$  lie within  $x \in \{-\frac{\mu}{2}, -\frac{\mu}{2} + 1, \dots, \frac{\mu}{2} - 1, \frac{\mu}{2}\}$  and are distributed according to the

probability mass function

$$\Pr[x|x \leftarrow \beta_\mu] = \frac{\mu!}{(\mu/2 + x)!(\mu/2 - x)!} 2^{-\mu}.$$

The LPR scheme is based on the hardness of **Ring-LWE (RLWE)** [38], an algebraic extension of LWE, with samples of the form  $(a, b = a \cdot s + e)$  for polynomials  $a, s, e$ , all belonging to  $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ . The module variant **Module-LWE (MLWE)** [36] treats samples of the form  $(\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e})$  for  $\mathbf{A} \in R_q^{l \times l}$  and  $\mathbf{s}, \mathbf{e} \in R_q^l$ . LWR [9] adds the random error term implicitly in the second component of the LWE sample (instead of explicitly adding it) by scaling down from  $q$  to a smaller modulus  $p$ . Such a sample is of the form  $(a, b = \lfloor \frac{p}{q} a \cdot s \rfloor) \in \mathbb{Z}_q^k \times \mathbb{Z}_p$ . Ring variants as well as module variants also exist for LWR.

Algorithm 1 describes key generation in an LPR-based scheme. The public key is an RLWE-sample  $(a, b)$  and is based on the secret key  $s$ . Algorithm 2 illustrates encryption of an  $n$ -bit message  $m$  (after mapping it to  $R_q$ ): two additional LWE-samples are generated and  $m$  is added to the most significant bits of the second sample. Decryption (see Algorithm 3) multiplies the first sample by the secret key, then subtracts it from the second sample. A final decoding step removes the randomness introduced by the error terms  $e, e'$ , and  $e''$  from  $m'$ , yielding  $m$ .

In this article, we target the medium-security parameter sets of the MLWE-based KEM Kyber ( $l = 3, n = 256, q = 3,329$ , and  $\mu = 4$ ) and MLWR-based KEM Saber ( $l = 3, n = 256, q = 2^{13}, p = 2^{10}$ , and  $\mu = 8$ ). We note that the presented attacks can be easily extended to other security levels.

---

**ALGORITHM 1:** LPR key generation

---

- 1:  $a \leftarrow \mathcal{U}(R_q)$
  - 2:  $s, e \leftarrow \beta_\mu(R_q)$
  - 3:  $b = a \cdot s + e$
  - 4: **return**  $(pk = (a, b), sk = s)$
- 

---

**ALGORITHM 2:** LPR encryption: input  $(pk, m)$ 


---

- 1:  $s', e', e'' \leftarrow \beta_\mu(R_q)$
  - 2:  $b' = a \cdot s' + e'$
  - 3:  $v' = b \cdot s' + e'' + \lfloor \frac{q}{2} \rfloor m$
  - 4: **return**  $ct = (b', v')$
- 

---

**ALGORITHM 3:** LPR decryption: input  $(ct, sk)$ 


---

- 1:  $m' = v' - b' \cdot s$
  - 2: **return**  $\text{Decode}(m')$
- 

The chosen-plaintext attack-secure LPR-based PKEs can be converted to CCA-secure KEMs using a variant [26] of the FO [22, 23] transform. The transform can prevent attacks that make use of a so-called plaintext-checking oracle, which reveals information about the secret key by providing the decryption of specially crafted ciphertexts. Figure 1 shows the decapsulation process. The input ciphertext  $c$  is first decrypted, after which the obtained plaintext is re-encrypted, resulting in  $c'$ . The shared key is not revealed when a decryption failure occurs (i.e.,  $c \neq c'$ ). In other words, an attacker can no longer gain information about the secret key as maliciously chosen ciphertexts are rejected during decapsulation. However, as shown by Ravi et al., this approach does not mitigate CCAs that make use of side-channel leakages [48].

**2.2.2 NTRU.** NTRU-like schemes consider polynomial rings  $\mathbb{Z}_3[x]/\Phi_n$ ,  $\mathbb{Z}_q[x]/\Phi_n$ , and  $\mathbb{Z}_q[x]/(\Phi_1 \cdot \Phi_n)$ , where  $\Phi_1 = x - 1$  and  $\Phi_n = x^{n-1} + x^{n-2} + \dots + 1$ . Key generation, encryption, and decryption are described in Algorithms 4, 6, and 5, respectively. The routines `Sample` and `Lift` and sets  $L_r$  and  $L_m$  are explained in detail in [16]. The NTRU submission to the NIST PQC standardization procedure is a combination of NTRUEncrypt and NTRU-HRSS-KEM, and proposes parameter sets for both `ntruhs` and `ntruhrss`. NTRU's secret coefficients  $\{0, 1, 2\}$  are

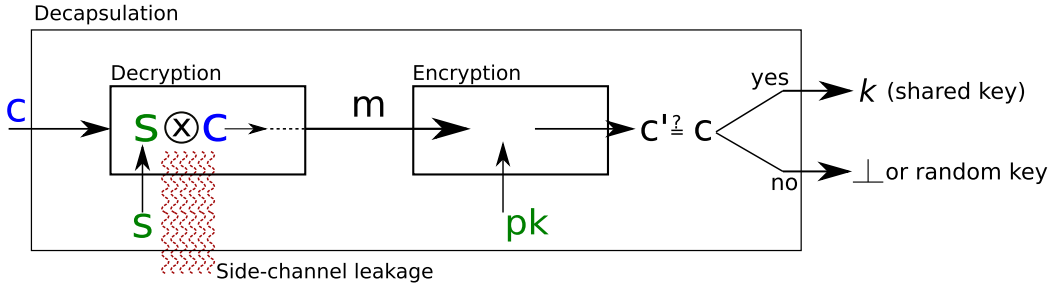


Fig. 1. The decapsulation procedure with re-encryption stage using the FO transform. The green- and blue-colored components respectively denote what the attacker can and cannot control during a chosen-ciphertext attack.

not binomially distributed: coefficients 0 occur with probability  $\frac{86}{256}$ , whereas 1 and 2 occur both with probability  $\frac{85}{256}$ .

---

**ALGORITHM 4:** NTRU key generation
 

---

- 1:  $f, g \leftarrow \text{Sample}()$
  - 2:  $f_q \leftarrow f^{-1} \bmod (q, \Phi_n)$
  - 3:  $h \leftarrow 3 \cdot g \cdot f_q \bmod (q, \Phi_1 \cdot \Phi_n)$
  - 4:  $h_q \leftarrow h^{-1} \bmod (q, \Phi_n)$
  - 5:  $f_p \leftarrow f^{-1} \bmod (3 \cdot \Phi_n)$
  - 6: **return**  $(pk = h, sk = (f, f_p, h_q))$
- 

---

**ALGORITHM 5:** NTRU decryption: input  $(ct, sk)$ 


---

- 1: **if**  $c \neq 0 \bmod (q, \Phi_1)$  **then return fail**
  - 2:  $a \leftarrow (c \cdot f) \bmod (q, \Phi_1 \cdot \Phi_n)$
  - 3:  $m \leftarrow (a \cdot f_p) \bmod (3, \Phi_n)$
  - 4:  $m' \leftarrow \text{Lift}(m)$
  - 5:  $r \leftarrow ((c - m') \cdot h_q) \bmod (q, \Phi_n)$
  - 6: **if**  $(r, m) \in L_r \times L_m$  **then return**  $(r, m, 0)$
  - 7: **return**  $(0, 0, 1)$
- 

---

**ALGORITHM 6:** NTRU encryption: input  $(pk, (r, m))$ 


---

- 1:  $m' \leftarrow \text{Lift}(m)$
  - 2:  $c \leftarrow (r \cdot h + m') \bmod (q, \Phi_1 \cdot \Phi_n)$
  - 3: **return**  $ct = c$
- 

The initial NTRU submissions [16] also used the FO transform to achieve CCA security. Instead, the final-round submission of NTRU [15] uses the Saito-Xagawa-Yamakawa [52] transform (Algorithm 5, Line 1). In contrast to the FO transform, the Saito-Xagawa-Yamakawa transform does not require re-encryption, and thus greatly improves the efficiency of the cryptographic scheme. In this work we use side-channel analysis to target the polynomial multiplication directly, which makes the presented attacks independent of the used transform.

### 2.3 Multiplication Strategies

Lattice-based cryptosystems usually resort to either the Number Theoretic Transform ( $\mathcal{O}(n \log n)$ ) [45] or Toom-Cook/Karatsuba ( $\mathcal{O}(n^{1+\epsilon})$ ,  $0 < \epsilon < 1$ ) [18, 31, 54] for efficient multiplication of polynomials with  $n$  coefficients. Neither one of the two algorithms is unambiguously the fastest in *any* scenario, being characterized by operand size and target processor architecture, since asymptotic complexities include constant factors [5]. The core idea of both multiplication strategies is multiplication by point evaluation. Given two  $n$ -coefficient polynomials  $a(x), b(x) \in R_q$ ,

evaluated at  $2n - 1$  distinct points  $\{x_1, x_2, \dots, x_{2n-1}\} \in \mathbb{Z}_q$ , we can compute the coefficients of  $(a \cdot b)(x)$  by interpolation of the values  $(a \cdot b)(x_i) = a(x_i)b(x_i)$ ,  $1 \leq i \leq 2n - 1$ .

**2.3.1 Toom-Cook.** The base case of Toom-Cook-based [18, 54] multiplication is that of *Karatsuba*, [31] which we illustrate for two linear polynomials  $a(x) = a_0 + a_1 \cdot x$  and  $b(x) = b_0 + b_1 \cdot x$ . Three evaluation points suffice, namely  $x_1 = 0$ ,  $x_2 = 1$ , and  $x_3 = \infty$ . We then get  $a(0) = a_0$ ,  $a(1) = a_0 + a_1$ ,  $a(\infty) = a_1$ ,  $b(0) = b_0$ ,  $b(1) = b_0 + b_1$ ,  $b(\infty) = b_1$ . These point evaluations can be interpolated as  $(a \cdot b)(x) = a_0b_0 + (a_1b_0 + a_0b_1) \cdot x + a_1b_1 \cdot x^2 = a(0)b(0) + (a(1)b(1) - a(0)b(0) - a(\infty)b(\infty)) \cdot x + a(\infty)b(\infty) \cdot x^2$ . This interpolation by point evaluation reduces the number of scalar multiplications from four ( $a_0b_0$ ,  $a_1b_0$ ,  $a_0b_1$ ,  $a_1b_1$ ) to three ( $a(0)b(0)$ ,  $a(1)b(1)$ ,  $a(\infty)b(\infty)$ ). Karatsuba's strategy can be applied to polynomials of arbitrary (power-of-2) number of coefficients  $n$  by rewriting  $a(x) = a^0(x) + a^1(x) \cdot x^{\frac{n}{2}}$  and  $b(x) = b^0(x) + b^1(x) \cdot x^{\frac{n}{2}}$ , and similarly splitting the sub-polynomials  $a^0(x)$ ,  $a^1(x)$ ,  $b^0(x)$ ,  $b^1(x)$  until a threshold degree is reached. Below this threshold degree the naive schoolbook method is used for multiplication. The method of  $w$ -way Toom-Cook generalizes the idea of Karatsuba by splitting  $rw$ -coefficient polynomials into  $w$  parts and performing the multiplication for  $r$  coefficients, after which the results are recombined. Multiplying the sub-polynomials is often done by applying a number of Karatsuba layers, before resorting to schoolbook multiplication.

All Toom-Cook-based versions of Saber and NTRU use  $w = 4$ , with evaluation points  $x_1 = 0$ ,  $x_2 = 1$ ,  $x_3 = -1$ ,  $x_4 = 2$ ,  $x_5 = -2$ ,  $x_6 = 3$ ,  $x_7 = \infty$ . Saber further splits the resulting sub-polynomials by two Karatsuba-layers, after which a 16-coefficient schoolbook multiplication is performed. All NTRU versions follow a similar structure, but with four Karatsuba layers (except for `ntruhs2048509`, which uses only three) and varying schoolbook thresholds. To illustrate the four-way Toom-Cook procedure, we consider two  $n$ -coefficient polynomials  $a(x)$  and  $b(x)$  that are split into four "equally large" polynomials:  $a(y) = a^0(x) + a^1(x) \cdot y + a^2(x) \cdot y^2 + a^3(x) \cdot y^3$  and  $b(y) = b^0(x) + b^1(x) \cdot y + b^2(x) \cdot y^2 + b^3(x) \cdot y^3$ , with  $y = x^{\frac{n}{4}}$ . Out of the seven  $\frac{n}{4}$ -coefficient multiplications, denoted by  $(*)$ , at seven evaluation points as specified above, we consider the following four ones in our attack proposal in Section 3:

$$\text{mul}_1 = a(0) * (0) = a^0(x) * b^0(x) \quad (1)$$

$$\begin{aligned} \text{mul}_2 &= a(2) * b(2) \\ &= (a^0(x) + 2 \cdot a^1(x) + 4 \cdot a^2(x) + 8 \cdot a^3(x)) \\ &\quad * (b^0(x) + 2 \cdot b^1(x) + 4 \cdot b^2(x) + 8 \cdot b^3(x)) \end{aligned} \quad (2)$$

$$\begin{aligned} \text{mul}_3 &= a(3) * b(3) \\ &= (a^0(x) + 3 \cdot a^1(x) + 9 \cdot a^2(x) + 27 \cdot a^3(x)) \\ &\quad * (b^0(x) + 3 \cdot b^1(x) + 9 \cdot b^2(x) + 27 \cdot b^3(x)) \end{aligned} \quad (3)$$

$$\text{mul}_4 = a(\infty) * b(\infty) = a^3(x) * b^3(x). \quad (4)$$

**2.3.2 Number Theoretic Transform.** The NTT can be regarded as a special form of the **Fast Fourier Transform (FFT)**. Since it computes over  $\mathbb{Z}_q$  instead of  $\mathbb{C}$ , no floating-point operations are required. The NTT also facilitates polynomial multiplication by some form of point evaluation. If

$$q \equiv 1 \pmod{n}, \quad (5)$$

then a primitive  $n$ th root of unity  $\omega \in \mathbb{Z}_q$ , satisfying  $\omega^{n/2} \equiv -1 \pmod{q}$ , exists and  $x^n - 1$  can be factorized as  $\prod_{i=0}^{n-1} (x - \omega^i)$ . Given two polynomials  $a(x), b(x) \in R_q$  to be multiplied, the NTT



computes the point evaluations

$$\text{NTT}(a(x)) = (a(\omega^0), \dots, a(\omega^{n-1})) \quad \text{and} \quad \text{NTT}(b(x)) = (b(\omega^0), \dots, b(\omega^{n-1})).$$

Multiplication of  $a(x)$  and  $b(x)$  modulo  $x^n - 1$  is then given by  $\text{NTT}^{-1}(\text{NTT}(a(x)) \circ \text{NTT}(b(x)))$ , where  $\circ$  denotes coefficient-wise multiplication. If  $n = 2^k$ , a power of 2, then the NTT can be computed by iteratively factorizing the original polynomial  $x^n - 1$  “in half” down to linear degree. Each factorization step is given by the map

$$\frac{\mathbb{Z}_q[x]}{x^{n/2^i} - \zeta^2} \rightarrow \frac{\mathbb{Z}_q[x]}{x^{n/2^i} + \zeta} \times \frac{\mathbb{Z}_q[x]}{x^{n/2^i} - \zeta} : a_0(x) + a_1(x) \cdot x^{n/2^{i-1}} \mapsto (a_0(x) - \zeta \cdot a_1(x), a_0(x) + \zeta \cdot a_1(x)),$$

for some power  $\zeta$  of  $\omega$ , called a *twiddle factor*. This map is termed the *Cooley-Tukey butterfly* [19], and its inverse (up to a factor  $\frac{1}{2}$ ) the *Gentleman-Sande butterfly* [24].

Most LWE-based schemes multiply over a ring  $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ . To this end, we can exploit the fact that  $x^{2n} - 1 = (x^n + 1)(x^n - 1)$ ; hence, any root of  $x^n + 1$  is also a root of  $x^{2n} - 1$ . If a primitive  $2n$ th root of unity  $\omega_{2n}$  exists, then these shared roots can be written as the odd powers  $\omega_{2n}^{2j+1} = \omega_{2n} \cdot \omega_n^j$ , for  $j \in \{0, \dots, n-1\}$ , with  $\omega_n = \omega_{2n}^2$ . When multiplying two polynomials  $a(x)$  and  $b(x)$  over  $R_q$ , it suffices to compute the regular  $n$ -coefficient NTT on  $a(\omega_{2n} \cdot x)$  and  $b(\omega_{2n} \cdot x)$ . This procedure is called the *negacyclic NTT*, and it strengthens Equation (5) to

$$q \equiv 1 \pmod{2n}. \quad (6)$$

We assume from now on to be always working over rings of the form  $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ .

For efficiency reasons, one might want to stop the recursive splitting prematurely after  $l$  (with  $l < k$ ) layers, so that the component-wise multiplication is of modular degree at least 2. The resulting *incomplete NTT* generalizes Equation (6) to  $q \equiv 1 \pmod{\frac{n}{2^{k-l-1}}}$ , so that a primitive  $\frac{n}{2^{k-l-1}}$ th root of unity exists.

Some authors of LWE-based schemes choose to use a modulus  $q$  not satisfying Equation (6), as is the case for Saber and NTRU. To still facilitate NTT-based polynomial multiplication, we can represent the polynomial coefficients in  $[-\frac{q}{2}, \frac{q}{2})$  and choose a new modulus  $q'$  such that both  $q' > \frac{nq^2}{2}$  and Equation (6) hold [17]. After performing polynomial multiplication by the NTT over  $\mathbb{Z}_{q'}[x]$ , the resulting coefficients can then be reduced back to  $\mathbb{Z}_q[x]$ .

Among the lattice-based KEM finalists, NTRU and Saber were initially designed to use Toom-Cook/Karatsuba-based polynomial multiplications [28, 30, 33, 40]. However, Chung et al. [17] later proposed NTT-based multiplication for these schemes using a larger modulus as described in this section. The most efficient implementations of Kyber have always used NTT-based multiplication.

**2.3.3 Montgomery Reduction.** When performing the NTT, we should be careful to avoid register overflow, which may occur when multiplying a polynomial coefficient with (a power of) a twiddle factor [39]. Therefore, we must modularly reduce intermediate NTT values by  $q$  regularly enough. The *Montgomery modular multiplication* [42] can be used to implement 32-bit modular reduction efficiently in constant time [44] and is suitable for the 32-bit registers of the Cortex-M4 [17]. Montgomery reduction makes use of the *REDC algorithm*, which requires a value  $R > q$  such that  $\gcd(q, R) = 1$  and such that basic computations mod  $R$  are inexpensive. The algorithm provides an efficient way to compute  $TR^{-1} \pmod{q}$ , given  $T$ , if  $0 \leq T < Rq$  is satisfied, and requires modular reduction by  $R$  instead of  $q$ .

## 2.4 Correlation Power Analysis

The attacks covered in this article use the CPA method introduced by Brier et al. [14]. In essence, the attacker will attempt to recover the cryptographic key using a divide-and-conquer approach.



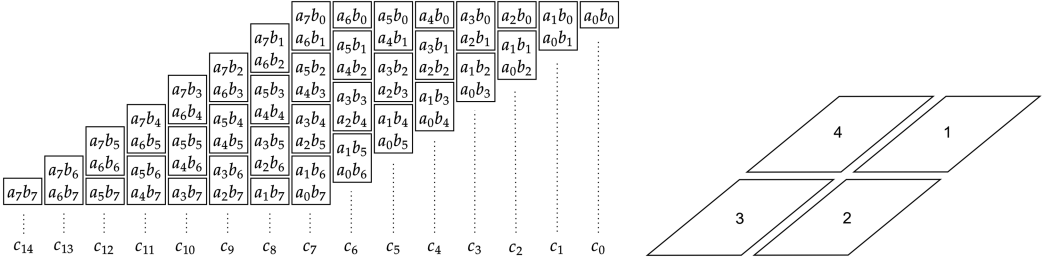


Fig. 2. Left: An illustration of the operations executed during 8-coefficient polynomial multiplication  $c(x) = \sum_{i=0}^{14} c_i \cdot x^i = a(x) \cdot b(x)$  of arbitrary polynomials  $a(x) = \sum_{i=0}^7 a_i \cdot x^i$  and  $b(x) = \sum_{i=0}^7 b_i \cdot x^i$ . Right: Sub-division of 16-coefficient schoolbook multiplication into four 8-coefficient multiplications according to specific order as indicated by the numbers in the separate sub-parallelograms [29].

A target device implements the target cipher and produces some (unintentional) side-channel leakage that is recorded by the attacker. A power model (e.g., Hamming weight or Hamming distance) is combined with a known input and unknown secret key. The attacker will guess the secret key by computing the correlation (e.g., using Pearson correlation) between the hypothetical power consumption and real side-channel measurements. Since attacks using power measurements and EM emanation are based on the same principles, we also consider the latter technique to be applicable to our proposed attacks.

In the remainder of this work we exploit side-channel leakage of the multiplication between the secret polynomial and a ciphertext polynomial during decryption, as indicated in Figure 1. Specifically, we target Line 1 in Algorithm 3 for LWE-based schemes and Line 2 in Algorithm 5 for NTRU-like schemes. We try to recover all  $n$  coefficients of the secret polynomials and we verify their correctness by decrypting the encryption of a known plaintext.

### 3 CORRELATION POWER ANALYSIS ON TOOM-COOK-BASED POLYNOMIAL MULTIPLICATION

In this section we propose a CPA approach targeting Toom-Cook-based multiplication in both Saber and NTRU. The attacks for both schemes follow the same main structure but differ slightly due to the integer ring to which the secret polynomial coefficients belong, as well as the schoolbook threshold. This section starts by providing details on the targeted schoolbook multiplication implementation, followed by the general structure of the attack and scheme-specific details.

#### 3.1 Schoolbook Multiplication Implementation

We target the basic schoolbook multiplication proposed by Kannwischer et al. in [29], which is used in multiple implementations of Saber and NTRU. The polynomial coefficients are all packed in pairs, which allows for efficient manipulation by the 32-bit ARMv7E-M instruction set. The latter includes parallel multiplications and instructions that can compute on combinations of low and high halfwords. Our attack targets the generic structure of the multiplication and is therefore independent of the target platform and instruction set. The implementation follows a parallelogram-like structure, as illustrated by Figure 2, to perform an eight-coefficient multiplication of a ciphertext polynomial  $a(x) = \sum_{i=0}^7 a_i x^i$  and secret polynomial  $b(x) = \sum_{i=0}^7 b_i x^i$ . Each column consists of all additive terms needed to compute  $c_i$ ,  $0 \leq i < 15$ , the coefficients of the resulting polynomial  $c(x) = \sum_{i=0}^{14} c_i x^i = a(x) \cdot b(x)$ . Each rectangle refers to the execution of one instruction. A rectangle containing two terms indicates that both operations are performed in parallel by a single instruction. The parallelogram-like structure can be generalized for an arbitrary number of

Table 1. Saber and NTRU Toom-Cook Parameter Sets

| Name          | $n$ | $q$              | Schoolbook Threshold(s) |
|---------------|-----|------------------|-------------------------|
| Saber         | 256 | $1,024 = 2^{10}$ | 16 (16)                 |
| ntruhs2048509 | 509 | $2,048 = 2^{11}$ | 16 (32)                 |
| ntruhs2048677 | 677 | $2,048 = 2^{11}$ | 10 (24), 11 (40)        |
| ntruhrss701   | 701 | $8,192 = 2^{13}$ | 11 (64)                 |
| ntruhs4096821 | 821 | $4,096 = 2^{12}$ | 12 (8), 13 (56)         |

coefficients of the multiplicands  $a(x)$  and  $b(x)$ . The order of operations is degree dependent: instructions in the same column are normally executed one after another, whereas neighboring columns might not be. We refer to the source code in the pqm4-library [30] for the ordering for all relevant degrees.

We exploit the side-channel leakage that results from storing the coefficients  $c_i$ . Note that, for any  $m$ -coefficient polynomial multiplication, every secret coefficient  $b_i$ ,  $0 \leq i < m$ , is involved in one scalar multiplication per resulting coefficient  $c_i, c_{i+1}, \dots, c_{i+m-1}$ . It thus suffices to only target the computation of  $c_0, \dots, c_{m-1}$  to recover all coefficients  $b_i$ ,  $0 \leq i < m$ . Our strategy is then to guess each coefficient  $b_i$ ,  $0 \leq i < m$ , in that order, based on the value of  $c_i$ , so that we need to already know the values of  $b_0, \dots, b_{i-1}$ , taking into account the parallelogram-like structure.

Table 1 lists the parameter sets deployed by Saber and NTRU for Toom-Cook-based polynomial multiplication. The schoolbook thresholds vary from 10 to 16. The fourth column of the table also lists in brackets the number of sub-polynomials that each original polynomial splits into at the schoolbook level. It can be computed as  $\approx \frac{n}{\text{schoolbook threshold}}$ . This is the number of schoolbook multiplications that we target.

### 3.2 Generic Structure of the Known-ciphertext Attack

**3.2.1 Attacker Model.** We assume that the attacker has physical access to a device performing a decapsulation procedure using a long-term key. Additionally, the attacker can submit as many valid ciphertexts as he or she wants. The attacker is not required to know the output of the decapsulation, but passively records side-channel information.

**3.2.2 General Outline.** The core idea of our attack is to target various executions of schoolbook multiplication, as many as there are sub-polynomials into which the original secret polynomial splits at the schoolbook threshold degree. The recovered sub-polynomials are combined to retrieve the original  $n$ -coefficient polynomial.

Recalling our previous discussion on four-way Toom-Cook, we consider each of Equations (1), (2), (3), and (4), which all multiply two  $\frac{n}{4}$ -coefficient polynomials. These multiplications usually involve several Karatsuba layers, followed by the targeted schoolbook multiplication at the threshold degree. For example, Saber uses  $n = 256$  with two Karatsuba layers and schoolbook threshold degree 16. For this case the structure for Equation (1) is shown in Figure 3: there are nine multiplications at the threshold degree, but it suffices to target the four (non-composite) ones led to by the full arrows, since the total of 64 secret coefficients  $b_{:16}^0(x), b_{16:32}^0(x), b_{32:48}^0(x), b_{48:}^0(x)$  can be recovered from these. Similarly, we recover the 64 coefficients of  $b^3(x)$  directly from Equation (4). We then combine our knowledge of  $b^0(x)$  and  $b^3(x)$  in order to recover  $b^0(x) + 2 \cdot b^1(x) + 4 \cdot b^2(x) + 8 \cdot b^3(x)$  from Equation (2), and  $b^0(x) + 3 \cdot b^1(x) + 9 \cdot b^2(x) + 27 \cdot b^3(x)$  from Equation (3). Finally,  $b^1(x)$  and  $b^2(x)$  are recovered by solving a linear system.

**3.2.3 Attack on Schoolbook Multiplication.** We assume to be given a set of  $n$ -coefficient ciphertext polynomials  $a(x) = \sum_{j=0}^{n-1} a_j x^j$  and their corresponding decapsulation side-channel

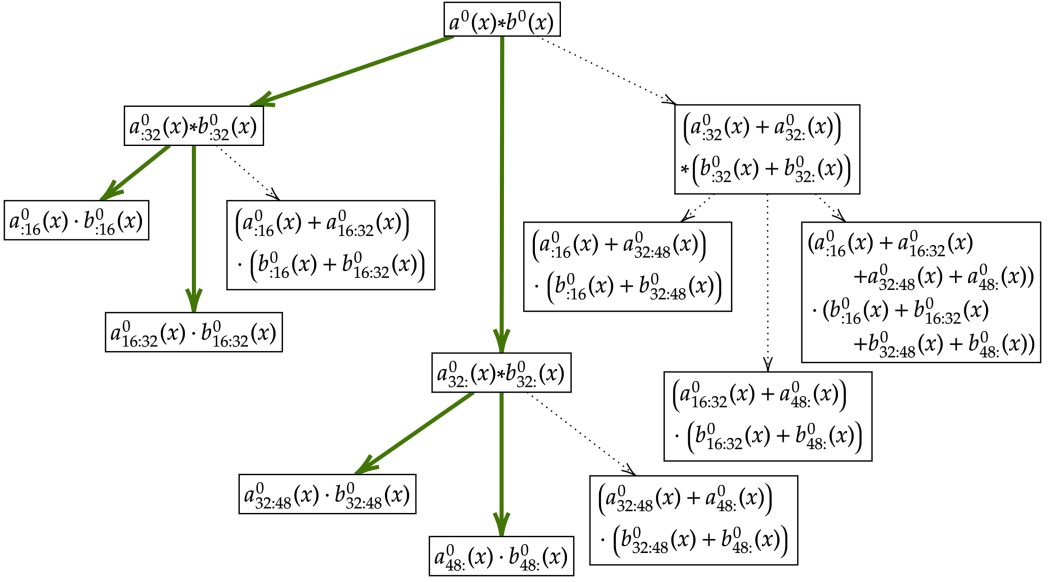


Fig. 3. Splitting structure of 64-coefficient polynomials in Saber. The full green lines denote the multiplications exploited in our attack. Polynomial multiplications to be split are denoted by (\*), schoolbook multiplications by (·).

measurements. We also assume that all of these side-channel measurements are captured from a device with a fixed secret polynomial  $b(x) = \sum_{j=0}^{n-1} b_j x^j$ . Our goal is to recover all coefficients  $b_{m \cdot k}, \dots, b_{m \cdot (k+1)-1}$ , for all  $0 \leq k < \frac{n}{m}$ , when the schoolbook multiplication  $(\sum_{i=0}^{m-1} a_{m \cdot k+i} x^i) \cdot (\sum_{i=0}^{m-1} b_{m \cdot k+i} x^i)$  is being executed. For ease of notation, we will from now on denote  $a_0 := a_{m \cdot k}, \dots, a_{m-1} := a_{m \cdot (k+1)-1}$  and  $b_0 := b_{m \cdot k}, \dots, b_{m-1} := b_{m \cdot (k+1)-1}$  for a certain chosen  $0 \leq k < \frac{n}{m}$ . For each value of  $k$ , we first perform a structured CPA strategy, called *CPA from right to left*, on the schoolbook multiplication parallelogram, resulting in a set of guesses for  $b_0, b_1, \dots, b_{m-1}$ . Some of the secret coefficients can not be recovered using CPA. For example, when  $b_0 = \dots = b_i = 0$  for some  $0 \leq i < m$ , the intermediate target values  $c_0, \dots, c_i$  will always be zero, regardless of the input ciphertext coefficient. We propose an iterative approach to rectify incorrect guesses of the secret polynomial coefficients. Our approach shares some similarities with the attack presented by Aysu et al. in [8], which applies horizontal DPA for secret key recovery of the key exchange protocols NewHope and Frodo. Both works focus on extraction of secret coefficients during polynomial multiplication, exploiting dependence between intermediate computations and removing false positives in the process. However, our attacks are not limited to a single measurement and are specifically designed to target the optimized implementations provided by the pqm4-library.

*CPA from right to left:* The name of this procedure refers to the direction in which we traverse the schoolbook multiplication parallelogram (recall the parallelogram from Figure 2). The core idea is, given  $b_0, \dots, b_{i-1}$  for some  $0 \leq i < m$ , to determine  $b_i$  as the value  $b_{\text{guess}}$  giving the maximum correlation between the side-channel traces and the Hamming weight of

$$c_i = a_i b_0 + \dots + a_1 b_{i-1} + a_0 b_{\text{guess}}. \quad (7)$$

If the resulting correlation coefficient is below a threshold  $r_{\text{min}}$ , we set  $b_{\text{guess}} = 0$ . This forces the subsequent guesses for  $b_{i+1}, \dots, b_{m-1}$  to be wrong (unless  $b_i$  is actually 0), which we will be able to detect and correct later. The value of  $r_{\text{min}}$  is determined empirically and will depend on the

target platform and measurement setup. As a baseline we recommend taking the average value of the correlation coefficient for the entire trace and multiplying this by a safety margin of 1.5. More details on setting  $r\_min$  are provided in Section 5.2.

We chose to use a threshold  $r\_min$  to help cope with two scenarios in which the correlation coefficient for  $c_i$  can be close to zero. The first scenario occurs when  $b_0, \dots, b_i$  are all truly 0, as there will not be any value for  $c_i$  for which the resulting correlation coefficient is larger than  $r\_min$ . Secondly, if a wrong guess was made for one of the  $b_0, \dots, b_{i-1}$ , then  $c_i$  yields a low correlation for all possible values of  $b_{guess}$ . Selecting  $b_{guess}$  according to highest correlation will therefore most likely give an incorrect value of  $b_i$ . But then the ensuing guesses for  $b_{i+1}, \dots, b_m$  will be wrong as well (i.e., the error propagates). We propose one possible correction mechanism, which we refer to as the *iterative approach*, to detect and correct such invalid guesses that arise in this second scenario. The approach will reject a set of guessed coefficients  $b_0, \dots, b_{m-1}$  whenever the correlation for  $c_{m-1}$  is noticeably low.

*Iterative approach:* Given the regular structure of the multiplication, the correlation coefficients for the different secret coefficients are expected to be of the same magnitude. If the correlation for  $c_{m-1}$  is significantly lower than when  $b_{m-1}$  is guessed correctly, then  $b_i$ , for some  $0 \leq i < m-1$ , was incorrectly guessed. This must be due to  $b_0, \dots, b_i$  (a set of consecutive secret coefficients) having values that we can't recover simply by looking at correlation for  $c_0, \dots, c_i$  as we do in CPA from right to left. Such sets of values are what we call *unrecoverable values*. (The collection of sets of unrecoverable values depends on the target scheme; details are provided in Sections 3.3 and 3.4 for Saber and NTRU, respectively.) We propose the following strategy to fix this problem. If  $c_{m-1}$  has low correlation, we force the first  $i$  coefficients  $b_0, \dots, b_i$ , for some  $0 \leq i < m-1$ , to be set to such a set of unrecoverable values. More specifically, for increasing value of  $i$ , we enumerate all sets of unrecoverable values for  $b_0, \dots, b_i$ . After each step of this enumeration, we run the CPA from right to left again (but only for  $b_{i+1}, \dots, b_{m-1}$ ). We terminate this iterative process as soon as the correlation for  $b_{m-1}$  attains some minimal correlation value  $t\_min$ . We refer to Algorithm 7 for an algorithmic description. The value of  $t\_min$  is set in an empirical manner and will be discussed in Section 5.2.

---

**ALGORITHM 7:** CPA from right to left with iterative approach

---

- 1:  $S$  = set of unrecoverable values for  $[b_0, \dots, b_i]$ , ordered by increasing length  $i+1$ , for  $0 \leq i < m-1$
  - 2:  $k = 0$
  - 3: guess  $[b_0, \dots, b_{m-1}]$  using CPA from right to left
  - 4:  $c_{m-1} = a_{m-1}b_0 + \dots + a_0b_{m-1}$
  - 5: **while**  $\text{corr}_{c_{m-1}} < t\_min$  **do**
  - 6:      $s = S[k]$
  - 7:      $[b_0, \dots, b_{|s|-1}] = s$
  - 8:     guess  $[b_{|s|}, \dots, b_{m-1}]$  using CPA from right to left
  - 9:      $c_{m-1} = a_{m-1}b_0 + \dots + a_0b_{m-1}$
  - 10:     $k = k + 1$
  - 11: **return**  $[b_0, \dots, b_{m-1}]$
- 

We propose two different iterative approaches: one for  $b^0(x)$  and  $b^3(x)$  (“iterative approach without offset”), and one for  $b^0(x) + 2 \cdot b^1(x) + 4 \cdot b^2(x) + 8 \cdot b^3(x)$  and  $b^0(x) + 3 \cdot b^1(x) + 9 \cdot b^2(x) + 27 \cdot b^3(x)$  (“iterative approach with offset”). For further scheme-specific details of the iterative approaches, we refer to Sections 3.3 and 3.4.

Our choice of only checking the correlation for  $b_{m-1}$  is motivated by the observation of unavoidable error propagation once one of the  $b_i$ s is wrongly guessed. Moreover, we observed that, if all coefficients  $b_0, \dots, b_{i-1}$  are correctly guessed, then the correlation for the correct value  $c_i$  will increase for increasing value of  $i$ . If the correct key is still not recovered after the iterative approach, the attack should be repeated with either more traces or different values for  $r_{\min}$  and  $t_{\min}$ . Alternatively, a more complex attack strategy can be developed that combines the CPA from right to left with a CPA from left to right.

### 3.3 Attack on Saber

In the case of Saber, the ciphertext coefficients  $a_0, \dots, a_{15}$  lie within the range  $\{0, 1, \dots, 1022, 1023\}$ , whereas the secret coefficients  $b_0, \dots, b_{15}$  lie within  $\{0, 1, 2, 3, 4, 1020, 1021, 1022, 1023\}$ . Saber deploys a threshold degree 16 for schoolbook multiplication, of which the structure is split into four equally large eight-coefficient multiplications as illustrated in Figure 2. We target sub-parallelgram 1 for coefficients  $b_0, b_1, \dots, b_7$ , as well as sub-parallelgram 2 for  $b_8, \dots, b_{15}$ . We operate according to the following procedure:

- (1) Perform CPA from right to left and the iterative approach on sub-parallelgram 1, resulting in a set of guesses for  $b_0, b_1, \dots, b_7$ .
- (2) Perform CPA from right to left (without the iterative approach) on sub-parallelgram 2, resulting in a set of guesses for  $b_8, \dots, b_{15}$ .
- (3) Adjust the set of guesses from Step 1 based on the quality of the set of guesses from Step 2.

In the following paragraphs we clarify each of the necessary components for this procedure.

*Iterative approach:* For the scenario of “iterative approach without offset,” the cases  $b_0 = b_1 = \dots = b_{i-1} = 0$  and  $b_i \in \{1, 2, 4\}$  for some  $0 \leq i < 7$  will all give the same correlation for  $c_i$ . Recall from Equation (7) that  $c_i$  has the same Hamming weight for all of these cases. It is possible that one of the above cases occurred for a certain  $0 \leq i < 7$  if the correlation for  $c_7$  is not larger than  $t_{\min}$ . If this correlation check fails indeed, then we try out the other two cases for the same value of  $i$ , to see whether the CPA results for one of these might yield success instead. If  $t_{\min}$  is still not attained, we then consider the cases

- $b_0 = b_1 = \dots = b_i = 0$  and  $b_{i+1} \notin \{0, 1023\}$  for some  $0 \leq i < 7$ ,
- $b_0 = b_1 = \dots = b_{k-1} = 0$ ,  $b_k = 1023$ ,  $b_{k+1} = b_{k+2} = \dots = b_i = 0$ , and  $b_{i+1} \neq 0$  for some  $0 \leq k \leq i < 7$ ,

which will cause the values of  $c_0, \dots, c_i$  to have zero variance (but not  $c_{i+1}$ ). Thus, in these cases, we are unable to guess  $b_0, \dots, b_i$ . The iterative approach enumerates these cases of unrecoverable values, as explained before and illustrated in Algorithm 7, until an instance is found for which the correlation of  $c_7$  is at least  $t_{\min}$ .

We explain the procedure of “iterative approach with offset” for  $b^0(x) + 2 \cdot b^1(x) + 4 \cdot b^2(x) + 8 \cdot b^3(x)$ , which is analogous to the procedure for  $b^0(x) + 3 \cdot b^1(x) + 9 \cdot b^2(x) + 27 \cdot b^3(x)$ . For this iterative approach, we assume that all coefficients  $b^0(x)$  and  $b^3(x)$  have already been recovered. It thus remains to recover  $2 \cdot b^1(x) + 4 \cdot b^2(x)$ , of which the coefficients can take on any value  $2 \cdot r + 4 \cdot s$  with  $r, s \in \{0, 1, 2, 3, 4, 1020, 1021, 1022, 1023\}$ . We sort these values by decreasing probability, taking into account the centered binomial distribution of Saber’s secret coefficients. The value of  $b_0$  represents  $b_i^0 + 2 \cdot b_i^1 + 4 \cdot b_i^2 + 8 \cdot b_i^3$  for some  $i \in \{0, 16, 32, 48\}$  (recall Figure 3). If CPA from right to left gives a correlation less than  $t_{\min}$  for  $c_7$ , we set the value of  $2 \cdot b_i^1 + 4 \cdot b_i^2$  to be the first entry of the sorted list. This process is repeated for all entries in the sorted list until a value for  $c_7$  with higher correlation than  $t_{\min}$  is found. From our practical experiments it is clear

that only changing the value of  $b_0$  will eventually make the CPA succeed, and we don't have to make use of the tricky unrecoverable values as in the approach without offset.

We only perform an iterative approach for the guesses of  $b_0, \dots, b_7$ . When guessing  $b_i$  for some  $8 \leq i < 15$ , we are computing  $c_i$ , for which we already have an offset  $a_7 b_{i-7} + \dots + a_{i-7} b_7$ . Practice has shown that these offsets will result in correct guesses for  $b_8, \dots, b_{14}$ , removing the need for an iterative approach.

### 3.4 Attack on NTRU

For NTRU, the secret coefficients are in the range  $\{0, 1, 2\}$ , whereas the ciphertext coefficients are in the range  $[0, q)$ . Ambiguity will arise if all coefficients of a sub-polynomial are either 0 and 1, or 0 and 2, as the Hamming weight for each  $c_i$  in Equation (7) will be the same for both cases. This is not an unlikely scenario because of the distribution of NTRU's secret coefficients (see Section 2.2). Nevertheless, we can take this scenario into account when combining all our sub-polynomial guesses into the full polynomial. Depending on the value of  $n$ , different values for the schoolbook threshold are considered, as indicated by Table 1. We make a distinction between the several NTRU-versions based on whether the schoolbook threshold is less than 16 or equal to 16.

**3.4.1** *ntruhs2048677, ntruhrss701, and ntruhs4096821.* These three NTRU versions have a threshold degree less than 16, for which the schoolbook multiplication consists of one sub-parallellogram instead of four. Additionally, the order of operations has an "overlapping" structure regarding the indices of the  $c_i$ , that is,

$$c_1, c_3, c_5, \dots, c_{2(\text{threshold}-1)-1}, c_0, c_2, c_4, \dots, c_{2(\text{threshold}-1)} \quad (8)$$

for thresholds 10, 11, and 12, and

$$c_1, c_3, c_5, \dots, c_{2(\text{threshold}-1)-1}, c_{2(\text{threshold}-1)}, c_{2(\text{threshold}-1)-2}, \dots, c_4, c_2, c_0 \quad (9)$$

for threshold 13.

*Iterative approach:* Given the secret range  $\{0, 1, 2\}$ , when correlating against  $c_0 = a_0 \cdot b_0$  for  $b^0(x)$  or  $b^3(x)$ , we either get zero variance for the Hamming weight of  $c_0$  when  $b_0 = 0$  or indistinguishable variance when  $b_0 \in \{1, 2\}$ , as  $c_0$  will in both cases always have the same Hamming weight. In case the correlation exceeds the value of  $r\_min$ , we set our guess for  $b_0$  to 2 and we proceed with the CPA from right to left. If the value for the last coefficient  $b_{\text{threshold}-1}$  was not guessed with at least correlation  $t\_min$ , we try out the value  $b_0 = 1$ . In case this fails again, we permanently set  $b_0$  to 0. As before, this procedure of enumerating unrecoverable values is repeated for  $b_i$  for increasing the value  $0 \leq i < \text{threshold}$ , until  $t\_min$  is attained. We extend this "iterative approach without offset" by rejecting guesses of which locations within the traces do not agree with the specific ordering of Equations (8) or (9).

The "iterative approach with offset" is the same as for Saber, except that now the sorted list of values is much shorter so that the correct value will be found more quickly.

**3.4.2** *ntruhs2048509.* Since this version of NTRU uses the same schoolbook threshold as Saber, we can use the same overall strategy as applied in Saber, but using the iterative approach previously discussed for the other NTRU versions.

## 4 CORRELATION POWER ANALYSIS ON NTT-BASED POLYNOMIAL MULTIPLICATION

In this section, we propose a CPA approach targeting the NTT, for which we again exploit the underlying schoolbook multiplication. The main two differences with the CPA on Toom-Cook are that the ranges of the secret coefficients are now greatly magnified through the NTT, whereas



Table 2. Kyber, Saber, and NTRU NTT Parameter Sets

| Name          | NTT <sub>n</sub> | q (or q')  | Schoolbook Threshold |
|---------------|------------------|------------|----------------------|
| Kyber768      | 256              | 3,329      | 2                    |
| Saber         | 256              | 25,166,081 | 3                    |
| ntruhs2048509 | 1,024            | 1,043,969  | 4                    |
| ntruhs2048677 | 1,536            | 1,389,569  | 3                    |
| ntruhrss701   | 1,536            | 5,747,201  | 3                    |
| ntruhs4096821 | 1,728            | 3,365,569  | 3                    |

the schoolbook threshold degree is smaller. Table 2 lists the moduli, as well as the schoolbook threshold degrees, deployed by the NTT-based implementations of Kyber, Saber, and NTRU. Kyber is the only KEM to originally have an NTT-friendly modulus. Note that NTT<sub>n</sub>, the size of the NTT, is chosen to efficiently facilitate an NTT and is always at least as large as  $n$ .

#### 4.1 Schoolbook Multiplication Implementation

Even though most NTT-based multiplications used in different PQC schemes deploy unique NTT structures, the implementation of the schoolbook multiplication is always similar. We illustrate the concept for threshold degree 3 (as in ntruhs4096821), which can be easily generalized for arbitrary degrees. Given are two polynomials in the NTT domain  $\hat{a}(x) = \hat{a}_0 + \hat{a}_1x + \hat{a}_2x^2$  and  $\hat{b}(x) = \hat{b}_0 + \hat{b}_1x + \hat{b}_2x^2$ , defined over  $R_q = \mathbb{Z}_q[x]/(x^3 - \zeta)$  for some power of a twiddle factor  $\zeta$ . Modular multiplication of these two polynomials yields

$$\begin{aligned}
 \hat{a}(x) \cdot \hat{b}(x) &\equiv (\hat{a}_0 + \hat{a}_1x + \hat{a}_2x^2) \cdot (\hat{b}_0 + \hat{b}_1x + \hat{b}_2x^2) \bmod x^3 - \zeta \\
 &= (\hat{a}_0\hat{b}_0 + \zeta(\hat{a}_1\hat{b}_2 + \hat{a}_2\hat{b}_1)) + (\hat{a}_0\hat{b}_1 + \hat{a}_1\hat{b}_0 + \zeta\hat{a}_2\hat{b}_2)x + (\hat{a}_0\hat{b}_2 + \hat{a}_1\hat{b}_1 + \hat{a}_2\hat{b}_0)x^2 \\
 &= \hat{c}_0 + \hat{c}_1x + \hat{c}_2x^2.
 \end{aligned} \tag{10}$$

For the source code implementing these computations, we refer to the pqm4-library [30]. The computation of  $\hat{c}_0$  involves the addition  $\hat{a}_1\hat{b}_2 + \hat{a}_2\hat{b}_1$ , followed by a Montgomery multiplication with  $\zeta$ , after which  $\hat{a}_0\hat{b}_0$  is added. The coefficient  $\hat{c}_1$  is computed in a similar manner. No Montgomery multiplication by  $\zeta$  is needed when computing  $\hat{c}_2$ . For efficiency reasons involving the inverse NTT, one last Montgomery reduction is applied to the final values of all three coefficients  $\hat{c}_0$ ,  $\hat{c}_1$ ,  $\hat{c}_2$ , before being stored consecutively. We will target the Hamming weight of the first resulting coefficient being stored and the Hamming distance between consecutively stored coefficients, the order of which depends on the target implementation.

#### 4.2 Generic Structure of the Side-channel Attack

**4.2.1 Attacker Model.** We assume the same attacker model as for our CPA on Toom-Cook-based multiplication. The only difference is that we cannot simply use any random, valid ciphertexts for our attacks on NTRU's and Saber's NTT.

For NTRU, while targeting an  $m \times m$  schoolbook multiplication, we would need to consider all possible combinations of the  $m$  secret values involved for each of the  $m$  resulting coefficients. We refer to the second line in Equation (10), which shows how to compute the  $m$  resulting values for the case of ntruhs4096821. The number of combinations  $(q')^m$  increases with the value of the modulus  $q'$ . We propose a strategy that eliminates the need to guess these combinations for NTRU in Section 4.4. As Saber uses an even larger modulus  $q'$ , we will resort to guessing the secret coefficients byte per byte, which reduces our search space to  $[0, 2^8)$ . This attack strategy is

explained in more detail in Section 4.5 and requires ciphertexts of which the NTT coefficients can take on any 32-bit value.

**4.2.2 General Outline.** We are given a size- $\text{NTT}_n$  NTT that uses  $m$ -coefficient schoolbook multiplication. That is, after applying the necessary NTT layers on two  $n$ -coefficient polynomials  $a(x)$  and  $b(x)$ , there are  $\frac{\text{NTT}_n}{m}$  schoolbook multiplications to be performed of the form

$$\begin{aligned} & (\hat{a}_{m \cdot k} + \dots + \hat{a}_{m \cdot k + m - 1} x^{m-1}) \cdot (\hat{b}_{m \cdot k} + \dots + \hat{b}_{m \cdot k + m - 1} x^{m-1}) \bmod (x^m - \zeta_k) \\ &= \hat{c}_{m \cdot k} + \dots + \hat{c}_{m \cdot k + m - 1} x^{m-1} \end{aligned}$$

for some power of a twiddle factor  $\zeta_k$  and  $0 \leq k < \frac{\text{NTT}_n}{m}$ . Our aim is to target each of these  $\frac{\text{NTT}_n}{m}$  schoolbook multiplications to retrieve all coefficients  $\hat{c}_{m \cdot k + i}$ ,  $0 \leq k < \frac{\text{NTT}_n}{m}$ ,  $0 \leq i < m$ . To check the correctness of our guesses, we simply apply the inverse NTT and check whether the result matches the coefficients of  $c(x) = a(x) \cdot b(x) \bmod x^n + 1$ .

**4.2.3 Attack on Schoolbook Multiplication.** As mentioned in Section 4.1, given a schoolbook multiplication characterized by some index  $0 \leq k < \frac{\text{NTT}_n}{m}$ , we target the storage of all resulting values  $\hat{c}_{m \cdot k + i}$ ,  $0 \leq i < m$ . The final Montgomery reduction reduces the coefficients  $\hat{c}_{m \cdot k + i}$  to the interval  $(-\frac{q}{2}, \frac{q}{2}]$ . As explained later in Section 4.6, it suffices to consider this same range for the coefficients  $\hat{b}_{m \cdot k + i}$ . We select the value of  $\hat{b}_{m \cdot k + i}$  with the highest correlation for  $\hat{c}_{m \cdot k + i}$ .

*Iterative approach:* As for the attack on Toom-Cook-based multiplication, we can use an iterative approach to correct invalid guesses. Only for relatively small moduli, such as  $q = 3,329$  for Kyber, we do not explicitly implement an iterative approach, as we can already iterate over all possible combinations for  $(\hat{c}_{2k}, \hat{c}_{2k+1})$ ,  $0 \leq k < 128$ , in a reasonable amount of time. For larger moduli, as in the NTRU-based schemes, the range of possible values is much larger, so that we can no longer consider all combinations, as further explained in Section 4.4. The same holds for Saber.

### 4.3 Attack on Kyber

Kyber is the only KEM in this article using a relatively small NTT modulus. Since  $n = \text{NTT}_n = 256 = 2^8$  and  $q = 3329 = 13 \cdot n + 1$ , Kyber facilitates a seven-layer, incomplete NTT, followed by a two-coefficient schoolbook multiplication. In the implementation, the two resulting coefficients  $\hat{c}_{2k} = \hat{a}_{2k} \hat{b}_{2k} + \zeta \hat{a}_{2k+1} \hat{b}_{2k+1}$  and  $\hat{c}_{2k+1} = \hat{a}_{2k} \hat{b}_{2k+1} + \hat{a}_{2k+1} \hat{b}_{2k}$ , for  $0 \leq k < 128$ , are stored by the same instruction instead of consecutively. We thus need to guess two coefficients at once within the range  $(-\frac{q}{2}, \frac{q}{2}]$ , implying a search over  $q^2 = 11,082,241$  combinations. Iterating over this number of combinations can still be done efficiently. We simply select the pair of values resulting in the largest correlation coefficient as our key guess.

### 4.4 Attack on NTRU

As opposed to Kyber, the NTT moduli for all NTRU versions are rather large, on the order of  $2^{20}$  (recall Table 2). It thus becomes computationally intensive (though not impossible) to try out all possible combinations of the secret coefficients when analyzing the schoolbook multiplication. For a schoolbook threshold  $m$ , recovering the secret coefficients  $\hat{b}_{m \cdot k}, \dots, \hat{b}_{(m+1) \cdot k - 1}$  requires making  $\approx 2^{20 \cdot m}$  key guesses when correlating against the resulting  $\hat{c}_{m \cdot k}, \dots, \hat{c}_{(m+1) \cdot k - 1}$ , for any  $0 \leq k < \frac{\text{NTT}_n}{m}$ . This makes correlating against the Hamming weight of all possible key values nearly impossible.

An alternative approach simplifies the attack by choosing each ciphertext polynomial  $a(x)$  in such a way that only NTT coefficients of the form  $\hat{a}_{m \cdot k}$ ,  $0 \leq k < \frac{\text{NTT}_n}{m}$ , can be non-zero. This would imply that  $\hat{c}_{m \cdot k + i} = \hat{a}_{m \cdot k} \cdot \hat{b}_{m \cdot k + i}$ , for all  $0 \leq i < m$ ; hence, we need not be concerned with any of the Montgomery multiplications by a power of a twiddle factor  $\zeta$ . For such a set of

NTT coefficients with “regular” indices to be all zero, we can choose the corresponding original ciphertext coefficients to be zero. Such ciphertexts  $c$  will still pass the check in Line 1 of Algorithm 5. This approach will, however, lead to decryption failures, which can be detected by the device under attack.

We illustrate the specifications of the attack for the case of ntruhs4096821. After carefully selecting the ciphertext coefficients, we correlate against the Hamming weight of the resulting coefficients, each time they are stored during execution. Depending on the NTRU version, the number of consecutive storing operations varies. For example, ntruhs4096821 stores its resulting coefficients in groups of  $24 = 8 \cdot 3$  coefficients. The values  $\hat{c}_{24 \cdot k + i}$  are (mostly) stored sequentially for increasing value of  $0 \leq i < 24$ , so that we can correlate against the Hamming weight of  $\hat{c}_{24 \cdot k}$  when guessing its value. We then correlate against the Hamming distance between  $\hat{c}_{24 \cdot k}$  and  $\hat{c}_{24 \cdot k + 1}$  when guessing the latter coefficient, and so on, for all  $0 \leq k < 72$ . Hamming distance implies a dependency between two coefficients, in the sense that  $\hat{c}_{24 \cdot k + 1}$  can only be correctly guessed if  $\hat{c}_{24 \cdot k}$  was correctly guessed, and so on. We exploit this characteristic in the iterative approach. These observations are of course implementation specific: if a different implementation strategy stores (for example) intermediates, then our attack would be greatly simplified as we can target the individual  $\hat{c}_i$ s.

*Iterative approach:* As the range  $(-\frac{q'}{2}, \frac{q'}{2}]$  can become quite large, we prematurely stop our brute-force search over this interval once we guess a value for each  $\hat{c}_{24 \cdot k}$ ,  $0 \leq k < 72$ , with correlation exceeding a given threshold. The threshold can be established in a similar way as  $r\_min$  and  $t\_min$  in the CPA on Toom-Cook. We then continue by guessing  $\hat{c}_{24 \cdot k + 1}$ , based on our guess for  $\hat{c}_{24 \cdot k}$ . If no value can be found for  $\hat{c}_{24 \cdot k + 1}$  with high enough correlation, we backtrack to our guess for  $\hat{c}_{24 \cdot k}$ , eliminate it, and sweep over the remaining range of  $(-\frac{q'}{2}, \frac{q'}{2}]$  to find another guess for  $\hat{c}_{24 \cdot k}$ . We repeat this procedure until we find a guess for  $\hat{c}_{24 \cdot k + 1}$  with high enough correlation, then we proceed analogously for  $\hat{c}_{24 \cdot k + 2}$ , and so on.

#### 4.5 Attack on Saber

For Saber the modulus  $q'$  is 25,166,081, roughly 10 times larger than the moduli for NTRU. The increased range  $(-\frac{q'}{2}, \frac{q'}{2}]$  of values for the secret coefficients decreases the efficiency of some of the previous CPA approaches, as in the best-case scenario an attacker would have to iterate over roughly  $2^{20}$  key guesses. The exact number of key guesses that have to be enumerated depends on the implementation and leakage model. For the provided reference implementation we would have to recover four secret coefficients simultaneously, as the results of the  $4 \times 4$  schoolbook multiplication are stored consecutively. In practice, this means we would have to make roughly  $2^{80}$  guesses. To alleviate this issue we propose a different strategy that targets the scalar multiplication on a smaller level.

Consider two 32-bit values  $\hat{a} = \hat{a}_{[3]} \cdot 2^{24} + \hat{a}_{[2]} \cdot 2^{16} + \hat{a}_{[1]} \cdot 2^8 + \hat{a}_{[0]}$  (known) and  $\hat{b} = \hat{b}_{[3]} \cdot 2^{24} + \hat{b}_{[2]} \cdot 2^{16} + \hat{b}_{[1]} \cdot 2^8 + \hat{b}_{[0]}$  (secret) to be multiplied. This byte notation simplifies the following statements. If we choose  $\hat{a}_{[0]} = \hat{a}_{[1]} = \hat{a}_{[2]} = 0$ , then

$$\hat{a} \cdot \hat{b} \bmod 2^{32} = \hat{a}_{[3]} \cdot \hat{b}_{[0]} \bmod 2^8.$$

The right-hand side of the latter equation can take on at most  $2^8$  distinct values, so that we can guess  $\hat{b}_{[0]}$  in a reasonable amount of time. Next, if we choose  $\hat{a}_{[0]} = \hat{a}_{[1]} = 0$ , we observe that

$$\hat{a} \cdot \hat{b} \bmod 2^{32} = (\hat{a}_{[3]} \cdot \hat{b}_{[0]} \cdot 2^8 + \hat{a}_{[2]} \cdot (\hat{b}_{[1]} \cdot 2^8 + \hat{b}_{[0]})) \bmod 2^{16}.$$

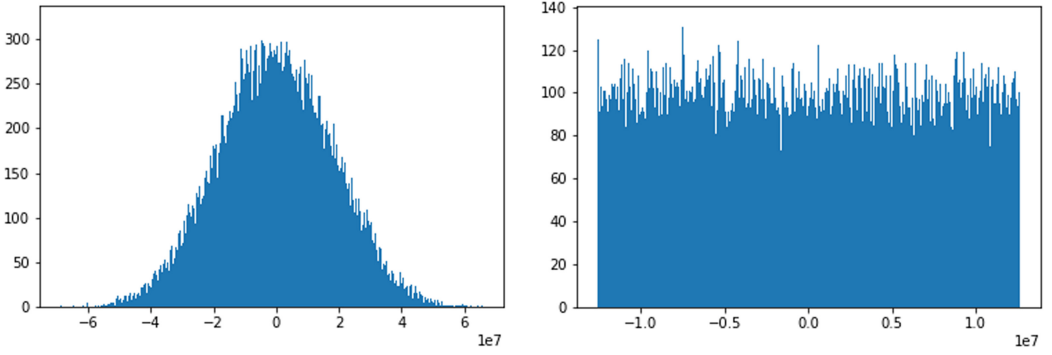


Fig. 4. Histograms illustrating the distribution of the secret NTT coefficients in Saber, unreduced (left) and reduced to  $(-\frac{q'}{2}, \frac{q'}{2}]$  (right),  $10^5$  samples, 1,000 bins.

Since we already know  $\hat{a}_{[3]}$ ,  $\hat{b}_{[0]}$ , and  $\hat{a}_{[2]}$ , the right-hand side of the latter equation can again only take on at most  $2^8$  distinct values, so we can guess  $\hat{b}_{[1]}$ . In an analogous way, we recover  $\hat{b}_{[2]}$  and then finally  $\hat{b}_{[3]}$ .

Note that we previously correlated against store operations for all CPAs, whereas we now consider the 32 least significant bits of the results of 32-bit multiplications. We found the individual scalar multiplications in

$$\zeta \cdot (\hat{a}_{4 \cdot k+3} \cdot \hat{b}_{4 \cdot k+2} + \hat{a}_{4 \cdot k+2} \cdot \hat{b}_{4 \cdot k+3}) + \hat{a}_{4 \cdot k+1} \cdot \hat{b}_{4 \cdot k} + \hat{a}_{4 \cdot k} \cdot \hat{b}_{4 \cdot k+1}, 0 \leq k < 64, \quad (11)$$

to yield the largest correlation against the traces of all four computations in Saber's  $4 \times 4$  school-book multiplication. The eight NTT coefficients are multiplied and subsequently accumulated in the same order as shown in Equation (11). We execute the (accumulated) CPA approach as explained above each time such a new scalar multiplication is performed. We use the Hamming-weight model when correlating against  $\hat{a}_{4 \cdot k+3} \cdot \hat{b}_{4 \cdot k+2}$ , and the Hamming-distance model for all ensuing operations. This attack strategy requires the attacker to submit invalid ciphertexts and can be detected by the device under attack if suitable countermeasures are implemented.

#### 4.6 A Note on the Binomial Distribution

Even if the polynomial coefficients of the original secret polynomial  $a(x)$  are restricted to a very small range, applying the NTT will greatly enlarge this range, of which the size is on the order of the modulus  $q$  (or  $q'$ ) times the number of NTT layers. The distribution of these NTT coefficients appears to be bell-shaped, as shown in Figure 4 on the left for Saber. However, all intermediate results in the computation of the (inverse) NTT are equivalent mod  $q$  (or mod  $q'$ ). Hence, we only need to consider reduced secret NTT values within, for example, the range  $(-\frac{q}{2}, \frac{q}{2}]$  (or  $(-\frac{q'}{2}, \frac{q'}{2}]$ ). We modularly reduce the bell-shaped distribution to this interval in the rightmost plot in Figure 4. This plot shows that there is no distinctive distribution in the new range, so we will consider the distribution to be uniform. We therefore do not take into account the original centered binomial distribution of the secret polynomial coefficients in our CPA.

## 5 RESULTS

This section covers the practical CPA results for both Toom-Cook and the NTT, as proposed in Sections 3 and 4, respectively. Complexity of the attacks is illustrated by means of number of traces used as well as runtime. The attacks are written in Python and executed on a MacBook with

Table 3. Saber and NTRU Toom-Cook CPA Runtimes

| Name          | Minimum Number of Traces | Average Runtime (s) |
|---------------|--------------------------|---------------------|
| Saber         | 200                      | 4                   |
| ntruhs2048509 | 400                      | 10                  |
| ntruhs2048677 | 100                      | 23                  |
| ntruhrss701   | 200                      | 32                  |
| ntruhs4096821 | 100                      | 38                  |

1.3 GHz Dual-Core Intel Core i5 processor. All pqm4 implementations were compiled using the arm-none-eabi-gcc compiler. These implementations are mostly written in C, but large parts of the Toom-Cook and NTT-based polynomial multiplication are written in assembly.

### 5.1 Measurement Setup

Our attack results are based on the code provided by the pqm4-library [30], a common framework for implementations and benchmarks of the NIST PQC finalists on the ARM Cortex-M4 microprocessor, which is equipped with the 32-bit ARMv7E-M architecture. This resource-constrained platform is often used for efficiency evaluation of cryptographic implementations due to its large enough memory to support public-key algorithms, while still being reasonably small and cheap [33].

Side-channel measurements were acquired from a NewAE CW308 UFO board combined with a STM32F415 (Cortex-M4) target board. The instantaneous power consumption was amplified using a Mini-Circuits ZFL-1000LN+ amplifier, low-pass filtered (20 MHz) and sampled by a Tektronix DPO7254C oscilloscope using a sample rate of 200 MS/s. The STM32F415 microcontroller was configured to use an 8 MHz external clock and 24 MHz core operating frequency. We selected the 24 MHz core operating frequency because it is one of the clock frequencies used by the pqm4 library to benchmark the implementations. The external clock was provided by a signal generator that shared a reference clock with the oscilloscope.

### 5.2 CPA on Toom-Cook

The acquired side-channel traces cover the polynomial multiplication, but not the full decapsulation, to reduce the runtime of the attack. The trace length increases with the value of  $n$ : we use traces of  $3.5 \cdot 10^5$  samples for  $n = 256$  (Saber) and  $5 \cdot 10^6$  samples for  $n = 821$  (ntruhs4096821). To further reduce the overall runtime of the attack, we first locate each schoolbook multiplication and segment the input trace. In this way the Pearson correlation coefficient can be calculated over short trace segments instead of the full trace. We locate a schoolbook multiplication by searching the moment in time where the first non-zero  $b_i$ ,  $0 \leq i < m$ , is involved in the computation of  $c_i$ . As  $b_0 = \dots = b_{i-1} = 0$ , there is only a small range of values that  $c_i = a_0 \cdot b_i$  can take on (depending on the range of  $b_i$ ), for each of which we correlate against the traces. The instance with highest correlation gives us a reference point in time at which the schoolbook multiplication is performed, and also provides us with a reference for selecting  $r_{\min}$  and  $t_{\min}$ . The value of  $r_{\min}$  is set to a generous lower bound on these correlation values, for which we chose 0.05. For  $t_{\min}$  we consider an average of the correlations. Regarding Saber and ntruhs2048509 (which both use a threshold degree 16), we set  $t_{\min}$  to 0.4, whereas a more generous value 0.5 is considered for the other NTRU-based schemes.

*Saber and NTRU.* Table 3 lists the minimum number of traces for the CPA on Toom-Cook for full recovery of a secret polynomial as well as the average runtimes. On average, the required

Table 4. NTRU NTT CPA Runtimes

| Name          | Average Runtime<br>per Group (s) | Number of<br>Elements per Group | Number of<br>Groups |
|---------------|----------------------------------|---------------------------------|---------------------|
| ntruhs2048509 | 300                              | 16 (4 sets of 4)                | 64                  |
| ntruhs2048677 | 282                              | 3                               | 512                 |
| ntruhrss701   | 1,040                            | 3                               | 512                 |
| ntruhs4096821 | 9,840                            | 24 (8 sets of 3)                | 72                  |

number of traces decreases for a smaller schoolbook threshold and for a lower value of  $n$ . The average runtime increases with  $n$ . Moreover, the runtime is affected by the average duration of the iterative approach for each sub-polynomial, which will depend on the **signal-to-noise ratio (SNR)** of the measurements and the selected  $r_{\min}$  and  $t_{\min}$ .

The proposed approach can be easily adapted for other versions of Saber, such as the higher-security FireSaber that uses a reduced set of secret coefficients  $\{0, 1, 2, 3, 1021, 1022, 1023\}$ . In fact, this reduced set of secret coefficients would speed up the iterative approach, resulting in lower overall runtime. For NTRU, the runtimes only consider a single full run of guessing all coefficients, iterative approach included. This does not include testing for combinations with “shifted zeros” for  $b^0(x)$  and  $b^3(x)$ , which would require new guesses for  $b^0(x) + 2 \cdot b^1(x) + 4 \cdot b^2(x) + 8 \cdot b^3(x)$  and  $b^0(x) + 3 \cdot b^1(x) + 9 \cdot b^2(x) + 27 \cdot b^3(x)$ . The number of such cases is relatively small, resulting in a minor impact on the overall runtime.

### 5.3 CPA on NTT

Note that guessing groups of NTT coefficients can be easily parallelized, since individual groups can be guessed independently. The runtimes provided in Table 4 were obtained using sequential recovery without parallelization. The side-channel traces cover the polynomial multiplication, and the number of samples per trace ranges from  $2 \cdot 10^4$  for Kyber to  $5 \cdot 10^5$  for ntruhs4096821.

The following technique was applied to reduce the number of time samples used during the CPA attack. Schoolbook multiplication is implemented in such a way that each of the  $m \times m$  schoolbook multiplications of two pairs of  $m$  NTT coefficients is followed by the  $m \times m$  schoolbook multiplication of the next two pairs of  $m$  NTT coefficients. Depending on the implementation, resulting coefficients can be stored after each  $m \times m$  multiplication, or they can be stored after  $j$  such  $m \times m$  multiplications, for some  $j \geq 1$ . We consider all these  $j \cdot m$  coefficients being consecutively stored as a group. For each group, we can narrow down the approximate location within the measured traces of when the coefficients are being stored; we search for as many distinctive peaks in the variance as there are groups. Based on these variance peaks, we can restrict our CPA to much smaller ranges than the full trace domain. We illustrate our reasoning for the case of ntruhs4096821 with size-1728 NTT, schoolbook threshold degree 3, and eight  $3 \times 3$  schoolbook multiplications per group. We thus expect  $\frac{1728}{8 \cdot 3} = 72$  variance peaks, as shown in Figure 5, which plots the trace variance during the full schoolbook multiplication that takes place between the forward and inverse NTT.

*Kyber.* Our approach requires 200 traces to recover all  $\text{NTT}_n$  coefficients. Guessing one pair of coefficients takes roughly 5 minutes on average. Therefore, our attack on Kyber provides a considerable speedup compared to the CPA proposed in [32]. We used a brute-force approach where the coefficients resulting from the modular multiplication were guessed in their entirety. Therefore, we only provided valid ciphertexts as inputs to the decapsulation.

*NTRU.* We ran the CPA on NTRU’s NTT with each time a fixed number of 500 traces, due to the long runtime of the attack. We attacked NTRU using the alternative approach proposed



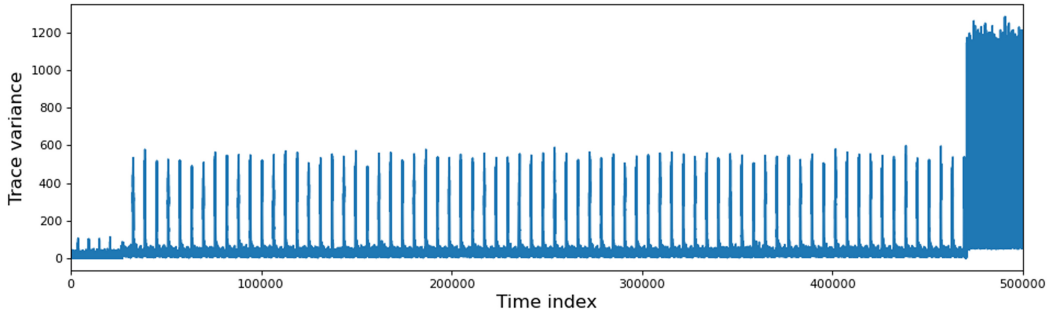


Fig. 5. Variance plot of 500 traces of schoolbook multiplication in ntruhs4096821, showing 72 peaks.

in Section 4.4 because of the high time complexity of performing an attack with only valid ciphertexts.

We considered the following values for the iterative approach threshold: 0.3 for ntruhs2048509 and ntruhs4096821, and 0.5 for ntruhs2048677 and ntruhrss701. We clearly see the influence of the size of the NTT modulus  $q'$  on the runtime per group of coefficients in Table 3. The larger the value of  $q'$ , the larger the range of values to brute-force over. Note that, unlike for Kyber, the runtimes also cover the iterative approach needed for corrections. Table 3 lists in the third column the number of elements contained in each independent group, as well as the number of groups in the fourth column.

*Saber.* Similar to NTRU, we applied the alternative approach when attacking Saber (see Section 4.5). In contrast to NTRU, we do not target the Hamming distance leakage of the full 32-bit state but target only 8 bits at a time. Targeting a smaller portion of the state reduces the SNR and results in a significant increase in the number of traces required to execute the attack. A minimum of 10,000 traces was required to mount a successful attack. Given the reduced number of target bits, the correct guess is not always ranked highest. We therefore enumerate all possible values for  $\hat{b}_{[0]}$ ; for each of these we then perform the proposed CPA to guess the remaining three bytes. Afterward, we select the full value  $\hat{b} = \hat{b}_{[3]} \cdot 2^{24} + \hat{b}_{[2]} \cdot 2^{16} + \hat{b}_{[1]} \cdot 2^8 + \hat{b}_{[0]}$  giving the highest correlation. Guessing one of the four bytes takes about 20s on average, which we have to repeat  $4 \cdot 256$  times per secret NTT coefficient.

## 6 CONCLUSION

In this article we propose a non-profiled side-channel attack methodology that targets polynomial multiplication. We apply our methodology to all lattice-based KEMs in the final round of the NIST PQC competition. In contrast to previous work, the attack methodology presented in this work is non-profiled and does not require purposefully crafted ciphertexts that result in decryption failures.

We target both Toom-Cook-based multiplication and NTT-based multiplication. Both of these polynomial multiplication strategies perform point-wise multiplication at a threshold level before the results are interpolated. We demonstrate how the secret operands involved in these scalar multiplications can be recovered using correlation power analysis. Our experiments show that Toom-Cook-based polynomial multiplication is more straightforward to attack. This is reflected by large differences in the execution time of attacks targeting Toom-Cook implementations and the attacks on implementations of the NTT-based multiplication. While we demonstrate our methodology for Kyber, Saber, and NTRU, it is also applicable to any other unprotected lattice-based scheme using Toom-Cook or the NTT for polynomial multiplication.

The proposed attacks can be mitigated by using masking and hiding countermeasures [11, 13]. Nevertheless, the overall attack methodology likely remains valid when targeting masked implementations if the adversary is able to combine the information from multiple samples.

## REFERENCES

- [1] M. Ajtai. 1996. Generating hard instances of lattice problems (extended abstract). In *Proceedings of the 28th Annual ACM Symposium on the Theory of Computing*. ACM, 99–108.
- [2] Gorjan Alagic, Jacob Alperin-Sheriff, Daniel Apon, David Cooper, Quynh Dang, John Kelsey, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, and Daniel Smith-Tone. 2017. Status Report on the Second Round of the NIST Post-Quantum Cryptography Standardization Process. Retrieved March 20, 2022, from <https://nvlpubs.nist.gov/nistpubs/ir/2020/NIST.IR.8309.pdf>. <https://nvlpubs.nist.gov/nistpubs/ir/2020/NIST.IR.8309.pdf>.
- [3] Martin R. Albrecht, Shi Bai, and Léo Ducas. 2016. A subfield lattice attack on overstretched NTRU assumptions - Cryptanalysis of some FHE and graded encoding schemes. In *Advances in Cryptology - 36th Annual International Cryptology Conference (CRYPTO'16), Proceedings, Part I (Lecture Notes in Computer Science)*, Matthew Robshaw and Jonathan Katz (Eds.), Vol. 9814. Springer, 153–178. [https://doi.org/10.1007/978-3-662-53018-4\\_6](https://doi.org/10.1007/978-3-662-53018-4_6)
- [4] Martin R. Albrecht, Rachel Player, and Sam Scott. 2015. On the concrete hardness of learning with errors. *J. Math. Cryptol.* 9, 3 (2015), 169–203. <http://www.degruyter.com/view/j/jmc.2015.9.issue-3/jmc-2015-0016/jmc-2015-0016.xml>.
- [5] Erdem Alkim, Dean Yun-Li Cheng, Chi-Ming Marvin Chung, Hülya Evkan, Leo Wei-Lun Huang, Vincent Hwang, Ching-Lin Trista Li, Ruben Niederhagen, Cheng-Jhih Shih, Julian Wälde, and Bo-Yin Yang. 2021. Polynomial multiplication in NTRU prime comparison of optimization strategies on Cortex-M4. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021, 1 (2021), 217–238. <https://doi.org/10.46586/tches.v2021.i1.217-238>
- [6] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. 2016. Post-quantum key exchange - A new hope. In *25th USENIX Security Symposium (USENIX Security'16)*, Thorsten Holz and Stefan Savage (Eds.). USENIX Association, 327–343. <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/alkim>.
- [7] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. 2021. CRYSTALS-Kyber. Algorithm Specifications and Supporting Documentation. (Round 3 Submission). Retrieved March 20, 2022, from <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210131.pdf>.
- [8] Aydin Aysu, Youssef Tobah, Mohit Tiwari, Andreas Gerstlauer, and Michael Orshansky. 2018. Horizontal side-channel vulnerabilities of post-quantum key exchange protocols. In *2018 IEEE International Symposium on Hardware Oriented Security and Trust (HOST'18)*. 81–88. <https://doi.org/10.1109/HST.2018.8383894>
- [9] Abhishek Banerjee, Chris Peikert, and Alon Rosen. 2012. Pseudorandom functions and lattices. In *Advances in Cryptology 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT'12), Proceedings (Lecture Notes in Computer Science)*, David Pointcheval and Thomas Johansson (Eds.), Vol. 7237. Springer, 719–737. [https://doi.org/10.1007/978-3-642-29011-4\\_42](https://doi.org/10.1007/978-3-642-29011-4_42)
- [10] Andrea Basso, Jose Maria Bermudo Mera, Jan-Pieter D'Anvers, Angshuman Karmakar, Sujoy Sinha Roy, Michiel Van Beirendonck, and Frederik Vercauteren. 2020. SABER: Mod-LWR based KEM (Round 3 Submission). Retrieved March 20, 2022, from <https://www.esat.kuleuven.be/cosic/pqcrypto/saber/files/saberspecround3.pdf>.
- [11] Michiel Van Beirendonck, Jan-Pieter D'Anvers, Angshuman Karmakar, Josep Balasch, and Ingrid Verbauwhede. 2021. A side-channel-resistant implementation of SABER. *ACM J. Emerg. Technol. Comput. Syst.* 17, 2 (2021), 10:1–10:26. <https://doi.org/10.1145/3429983>
- [12] Shivam Bhasin, Jan-Pieter D'Anvers, Daniel Heinz, Thomas Pöppelmann, and Michiel Van Beirendonck. 2021. Attacking and defending masked polynomial comparison for lattice-based cryptography. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021, 3 (2021), 334–359. <https://doi.org/10.46586/tches.v2021.i3.334-359>
- [13] Joppe W. Bos, Marc Gourjon, Joost Renes, Tobias Schneider, and Christine van Vredendaal. 2021. Masking Kyber: First- and higher-order implementations. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021, 4 (2021), 173–214. <https://doi.org/10.46586/tches.v2021.i4.173-214>
- [14] Eric Brier, Christophe Clavier, and Francis Olivier. 2004. Correlation power analysis with a leakage model. In *Cryptographic Hardware and Embedded Systems 6th International Workshop (CHES'04), Proceedings (Lecture Notes in Computer Science)*, Marc Joye and Jean-Jacques Quisquater (Eds.), Vol. 3156. Springer, 16–29. [https://doi.org/10.1007/978-3-540-28632-5\\_2](https://doi.org/10.1007/978-3-540-28632-5_2)
- [15] Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hülsing, Joost Rijneveld, John M. Schanck, Tsunekazu Saito, Peter Schwabe, William Whyte, Keita Xagawa, Takashi Yamakawa, and Zhenfei Zhang. 2020. NTRU Algorithm Specifications and Supporting Documentation, 2nd PQC Standardization Conference, 2019, University of California, Santa Barbara, CA. Retrieved September 30, 2021, from <https://ntru.org/f/ntru-20190330.pdf>.

- [16] Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hülsing, Joost Rijneveld, John M. Schanck, Peter Schwabe, William Whyte, and Zhenfei Zhang. 2019. NTRU Algorithm Specifications and Supporting Documentation, 2nd PQC Standardization Conference, 2019, University of California, Santa Barbara, CA. <https://ntru.org/f/ntru-20190330.pdf>.
- [17] Chi-Ming Marvin Chung, Vincent Hwang, Matthias J. Kannwischer, Gregor Seiler, Cheng-Jhih Shih, and Bo-Yin Yang. 2021. NTT multiplication for NTT-unfriendly rings new speed records for Saber and NTRU on Cortex-M4 and AVX2. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2021, 2 (2021), 159–188. <https://doi.org/10.46586/tches.v2021.i2.159-188>
- [18] S. A. Cook. 1966. *On the Minimum Computation Time of Functions*. Ph.D. Dissertation. Harvard University, 51–77.
- [19] James W. Cooley and John W. Tukey. 1965. An algorithm for the machine calculation of complex fourier series. *Math. Comp.* 19, 90 (1965), 297–301. <http://www.jstor.org/stable/2003354>.
- [20] Dana Dachman-Soled, Léo Ducas, Huijing Gong, and Mélissa Rossi. 2020. LWE with side information: Attacks and concrete security estimation. In *Advances in Cryptology - 40th Annual International Cryptology Conference (CRYPTO'20), Proceedings, Part II (Lecture Notes in Computer Science)*, Daniele Micciancio and Thomas Ristenpart (Eds.), Vol. 12171. Springer, 329–358. [https://doi.org/10.1007/978-3-030-56880-1\\_12](https://doi.org/10.1007/978-3-030-56880-1_12)
- [21] Jan Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. 2019. SABER: Mod-LWR based KEM, 2nd PQC Standardization Conference, 2019, University of California, Santa Barbara, CA. [https://www.esat.kuleuven.be/cosic/pqcrypto/saber/files/SABER\\_KEM\\_Round\\_2.zip](https://www.esat.kuleuven.be/cosic/pqcrypto/saber/files/SABER_KEM_Round_2.zip).
- [22] Eiichi Fujisaki and Tatsuaki Okamoto. 1999. Secure integration of asymmetric and symmetric encryption schemes. In *Advances in Cryptology (CRYPTO'99)*, Michael Wiener (Ed.). Springer, Berlin, 537–554.
- [23] Eiichi Fujisaki and Tatsuaki Okamoto. 2013. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology* 26, 1 (Jan. 2013), 80–101. <https://doi.org/10.1007/s00145-011-9114-1>
- [24] W. M. Gentleman and G. Sande. 1966. Fast fourier transforms: For fun and profit. In *Proceedings of the Fall Joint Computer Conference (AFIPS'66 (Fall))*. Association for Computing Machinery, New York, NY, 563–578. <https://doi.org/10.1145/1464291.1464352>
- [25] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. 1998. NTRU: A ring-based public key cryptosystem. In *Algorithmic Number Theory: 3rd International Symposium (ANTS-III), Proceedings*, Joe P. Buhler (Ed.). Springer, Berlin, 267–288. <https://doi.org/10.1007/BFb0054868>
- [26] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. 2017. A modular analysis of the Fujisaki-Okamoto transformation. In *Theory of Cryptography - 15th International Conference (TCC'17), Proceedings, Part I (Lecture Notes in Computer Science)*, Yael Kalai and Leonid Reyzin (Eds.), Vol. 10677. Springer, 341–371. [https://doi.org/10.1007/978-3-319-70500-2\\_12](https://doi.org/10.1007/978-3-319-70500-2_12)
- [27] Wei-Lun Huang, Jiun-Peng Chen, and Bo-Yin Yang. 2020. Power analysis on NTRU prime. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2020, 1 (2020), 123–151. <https://doi.org/10.13154/tches.v2020.i1.123-151>
- [28] Andreas Hülsing, Joost Rijneveld, John M. Schanck, and Peter Schwabe. 2017. High-speed key encapsulation from NTRU. In *Cryptographic Hardware and Embedded Systems 19th International Conference (CHES'17), Proceedings (Lecture Notes in Computer Science)*, Wieland Fischer and Naofumi Homma (Eds.), Vol. 10529. Springer, 232–252. [https://doi.org/10.1007/978-3-319-66787-4\\_12](https://doi.org/10.1007/978-3-319-66787-4_12)
- [29] Matthias J. Kannwischer, Joost Rijneveld, and Peter Schwabe. 2018. Faster multiplication in  $\mathbb{Z}_2^m[x]$  on Cortex-M4 to speed up NIST PQC candidates. Cryptology ePrint Archive, Report 2018/1018. (2018). <https://eprint.iacr.org/2018/1018>.
- [30] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. 2018. PQM4: Post-quantum crypto library for the ARM Cortex-M4. Retrieved March 20, 2022, from <https://github.com/mupq/pqm4>.
- [31] A. Karatsuba and Yu. Ofman. 1962. Multiplication of many-digital numbers by automatic computers. *Proceedings of USSR Academy of Sciences* 145, 7 (1962), 293–294.
- [32] Alexandre Karlov and Natacha Linard de Guertechin. 2021. Power analysis attack on Kyber. Cryptology ePrint Archive, Paper 2021/1311. (2021). <https://eprint.iacr.org/2021/1311>.
- [33] Angshuman Karmakar, Jose Maria Bermudo Mera, Sujoy Sinha Roy, and Ingrid Verbauwhede. 2018. Saber on ARM CCA-secure module lattice-based key encapsulation on ARM. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2018, 3 (2018), 243–266. <https://doi.org/10.13154/tches.v2018.i3.243-266>
- [34] Neal Koblitz. 1987. Elliptic curve cryptosystems. *Math. Comp.* 48, 177 (1987), 203–203. <https://doi.org/10.1090/s0025-5718-1987-0866109-5>
- [35] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. 1999. Differential power analysis. In *Advances in Cryptology 19th Annual International Cryptology Conference (CRYPTO'99), Proceedings (Lecture Notes in Computer Science)*, Michael J. Wiener (Ed.), Vol. 1666. Springer, 388–397. [https://doi.org/10.1007/3-540-48405-1\\_25](https://doi.org/10.1007/3-540-48405-1_25)
- [36] Adeline Langlois and Damien Stehlé. 2015. Worst-case to average-case reductions for module lattices. *Des. Codes Cryptogr.* 75, 3 (2015), 565–599. <https://doi.org/10.1007/s10623-014-9938-4>
- [37] Mun-Kyu Lee, Jeong Song, Dooho Choi, and Dong-Guk Han. 2010. Countermeasures against power analysis attacks for the NTRU public key cryptosystem. *IEICE Transactions* 93-A (2010), 153–163. <https://doi.org/10.1587/transfun.E93.A.153>

- [38] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. 2010. On ideal lattices and learning with errors over rings. In *Advances in Cryptology 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT'10), Proceedings*, Henri Gilbert (Ed.). Springer, Berlin, 1–23. [https://doi.org/10.1007/978-3-642-13190-5\\_1](https://doi.org/10.1007/978-3-642-13190-5_1)
- [39] Vadim Lyubashevsky and Gregor Seiler. 2019. NTTTRU: Truly fast NTRU using NTT. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2019, 3 (2019), 180–201. <https://doi.org/10.13154/tches.v2019.i3.180-201>
- [40] Jose Maria Bermudo Mera, Angshuman Karmakar, and Ingrid Verbauwhede. 2020. Time-memory trade-off in Toom-Cook multiplication: An application to module-lattice based cryptography. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2020, 2 (2020), 222–244. <https://doi.org/10.13154/tches.v2020.i2.222-244>
- [41] Victor S. Miller. 1986. Use of elliptic curves in cryptography. In *Advances in Cryptology (CRYPTO'85), Proceedings*, Hugh C. Williams (Ed.). Springer, Berlin, 417–426.
- [42] Peter L. Montgomery. 1985. Modular multiplication without trial division. *Math. Comp.* 44 (1985), 519–521.
- [43] NIST. 2017. Post-Quantum Cryptography Standardization. Retrieved October 10, 2021, from <https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Post-Quantum-Cryptography-Standardization>.
- [44] Peter Pessl and Robert Primas. 2019. More practical single-trace attacks on the number theoretic transform. In *LAT-INCRIPT (Lecture Notes in Computer Science)*, Vol. 11774. Springer, 130–149.
- [45] John M. Pollard. 1971. The fast Fourier transform in a finite field. *Math. Comp.* 25, 114 (1971), 365–374. <http://www.jstor.org/stable/2004932>.
- [46] Robert Primas, Peter Pessl, and Stefan Mangard. 2017. Single-trace side-channel attacks on masked lattice-based encryption. In *Cryptographic Hardware and Embedded Systems - 19th International Conference (CHES'17), Proceedings (Lecture Notes in Computer Science)*, Wieland Fischer and Naofumi Homma (Eds.), Vol. 10529. Springer, 513–533. [https://doi.org/10.1007/978-3-319-66787-4\\_25](https://doi.org/10.1007/978-3-319-66787-4_25)
- [47] Prasanna Ravi, Martians Frederic Ezerman, Shivam Bhasin, Anupam Chattopadhyay, and Sujoy Sinha Roy. 2021. Generic side-channel assisted chosen-ciphertext attacks on streamlined NTRU prime. *IACR Cryptol. ePrint Arch.* (2021), 718. <https://eprint.iacr.org/2021/718>.
- [48] Prasanna Ravi, Sujoy Sinha Roy, Anupam Chattopadhyay, and Shivam Bhasin. 2020. Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs. *IACR Trans. Cryptogr. Hardw. Embed. Syst.* 2020, 3 (2020), 307–335. <https://doi.org/10.13154/tches.v2020.i3.307-335>
- [49] Oded Regev. 2004. New lattice-based cryptographic constructions. *J. ACM* 51, 6 (2004), 899–942. <https://doi.org/10.1145/1039488.1039490>
- [50] Oded Regev. 2005. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC'05)*. ACM, 84–93.
- [51] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. 1978. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM* 21, 2 (1978), 120–126. <https://doi.org/10.1145/359340.359342>
- [52] Tsunekazu Saito, Keita Xagawa, and Takashi Yamakawa. 2018. Tightly-secure key-encapsulation mechanism in the quantum random oracle model. In *Advances in Cryptology - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT'18), Proceedings, Part III (Lecture Notes in Computer Science)*, Jesper Buus Nielsen and Vincent Rijmen (Eds.), Vol. 10822. Springer, 520–551. [https://doi.org/10.1007/978-3-319-78372-7\\_17](https://doi.org/10.1007/978-3-319-78372-7_17)
- [53] Joseph H. Silverman and William Whyte. 2007. Timing attacks on NTRUEncrypt via variation in the number of hash calls. In *Topics in Cryptology The Cryptographers' Track at the RSA Conference 2007 (CT-RSA'07), Proceedings (Lecture Notes in Computer Science)*, Masayuki Abe (Ed.), Vol. 4377. Springer, 208–224. [https://doi.org/10.1007/11967668\\_14](https://doi.org/10.1007/11967668_14)
- [54] A. L. Toom. 1963. The complexity of a scheme of functional elements realizing the multiplication of integers. In *Soviet Mathematics-Doklady*, Vol. 7. 714–716. <http://toomandre.com/my-articles/engmat/MULT-E.PDF>.
- [55] Zhuang Xu, Owen Pemberton, Sujoy Sinha Roy, and David F. Oswald. 2020. Magnifying side-channel leakage of lattice-based cryptosystems with chosen ciphertexts: The case study of Kyber. *IACR Cryptol. ePrint Arch.* (2020), 912. <https://eprint.iacr.org/2020/912>.

Received 1 April 2022; revised 7 August 2022; accepted 2 October 2022